Problem Solving Hackathon: Loops & Arrays (Complete with Test Cases)

Duration: Full Day (Morning to Evening)

Target: 100 Core Challenges with Input/Output Examples

Topics: Loops, Accumulators, Digit Manipulation, Patterns, 1D Arrays, Searching, Sorting, Two-Pointer Technique, Sliding Window, and 2D Matrices.

Section 1: Basic Loops & Digit Manipulation (Q1 – Q20)

1. ---

   Print numbers from 1 to N. (Input: N = 5 | Output: 1 2 3 4 5)
2. **Print numbers from N down to 1.** (Input: N = 5 | Output: 5 4 3 2 1)
3. **Sum of first N natural numbers.** (Input: N = 5 | Output: 15)
4. **Sum of all even numbers from 1 to N.** (Input: N = 6 | Output: 12)
5. **Multiplication table of N up to 10.** (Input: N = 3 | Output: 3 6 9 12 15 18 21 24 27 30)
6. **Count digits in an integer.** (Input: N = 54321 | Output: 5)
7. **Sum of digits of a number.** (Input: N = 1234 | Output: 10)
8. **Reverse an integer.** (Input: N = 1234 | Output: 4321)
9. **Check if a number is a Palindrome.** (Input: N = 1221 | Output: true; Input: N = 123 | Output: false)
10. **Find the first and last digit of a number.** (Input: N = 8243 | Output: First = 8, Last = 3)
11. **Compute base raised to power exponent (base^exp).** (Input: base = 2, exp = 4 | Output: 16)
12. **Find factorial of N.** (Input: N = 5 | Output: 120)

13. **Greatest Common Divisor (GCD) of two numbers.** (Input: a = 12, b = 18 | Output: 6)
14. **Least Common Multiple (LCM) of two numbers.** (Input: a = 4, b = 6 | Output: 12)
15. **Check if a number is Prime.** (Input: N = 17 | Output: Prime; Input: N = 15 | Output: Not Prime)
16. **Print all prime numbers up to N.** (Input: N = 15 | Output: 2, 3, 5, 7, 11, 13)
17. **Check Armstrong number.** (Input: N = 153 | Output: true; Input: N = 123 | Output: false)
18. **Check if a number is a Perfect Number.** (Input: N = 6 | Output: true (1 + 2 + 3 = 6))
19. **Binary to Decimal conversion.** (Input: "1101" | Output: 13)
20. **Decimal to Binary conversion.** (Input: 13 | Output: "1101")

Section 2: Patterns & Number Series (Q21 – Q35)

21. ---

    Right triangle star pattern of height N. (Input: N = 3 | Output: * \n ** \n ***)
22. **Inverted right triangle pattern of height N.** (Input: N = 3 | Output: *** \n ** \n *)
23. **Full pyramid star pattern of height N.** (Input: N = 3 | Output: * \n *** \n *****)
24. **Diamond star pattern of height 2N-1.** (Input: N = 3 | Output: * \n *** \n ***** \n *** \n *)
25. **Number triangle pattern (row i prints 1..i).** (Input: N = 3 | Output: 1 \n 1 2 \n 1 2 3)
26. **Floyd's Triangle of height N.** (Input: N = 3 | Output: 1 \n 2 3 \n 4 5 6)
27. **Pascal's Triangle up to N rows.** (Input: N = 4 | Output: [1], [1,1], [1,2,1], [1,3,3,1])
28. **Hollow square of size N.** (Input: N = 3 | Output: *** \n * * \n ***)
29. **Checkerboard pattern of 1s and 0s of size N x N.** (Input: N = 3 | Output: 1 0 1 \n 0 1 0 \n 1 0 1)
30. **Fibonacci series up to N terms.** (Input: N = 6 | Output: 0, 1, 1, 2, 3, 5)
31. **Print Nth Fibonacci number.** (Input: N = 7 | Output: 13)
32. **Collatz sequence starting at N.** (Input: N = 6 | Output: 6, 3, 10, 5, 16, 8, 4, 2, 1)
33. **Sum of series 1 + 1/2 + 1/3 + ... + 1/N.** (Input: N = 3 | Output: 1.8333)
34. **Hourglass star pattern of size N.** (Input: N = 3 | Output: ***** \n *** \n * \n *** \n *****)
35. **Strong number check (sum of factorials of digits equals number).** (Input: 145 | Output: true (1! + 4! + 5! = 145))

Section 3: 1D Array Fundamentals & Operations (Q36 – Q60)

36. ---

    Sum and Average of array elements. (Input: [1, 2, 3, 4, 5] | Output: Sum = 15, Avg = 3.0)
37. **Maximum and minimum elements.** (Input: [4, 9, 1, 7, 3] | Output: Max = 9, Min = 1)
38. **Second largest element without sorting.** (Input: [12, 35, 1, 10, 34, 1] | Output: 34)
39. **Count even and odd integers in array.** (Input: [1, 2, 3, 4, 5, 6] | Output: Even = 3, Odd = 3)
40. **Count positive, negative, and zero elements.** (Input: [-1, 2, 0, -3, 5, 0] | Output: Pos = 2, Neg = 2, Zero = 2)
41. **Reverse an array in-place.** (Input: [1, 2, 3, 4] | Output: [4, 3, 2, 1])
42. **Copy one array to another.** (Input: [10, 20, 30] | Output: [10, 20, 30])
43. **Insert element X at index K.** (Input: arr = [1, 2, 4, 5], X = 3, K = 2 | Output: [1, 2, 3, 4, 5])
44. **Delete element at index K.** (Input: arr = [10, 20, 30, 40], K = 1 | Output: [10, 30, 40])
45. **Frequency of each element in array.** (Input: [1, 2, 2, 3, 1, 4] | Output: {1: 2, 2: 2, 3: 1, 4: 1})
46. **Print unique elements in array.** (Input: [1, 2, 2, 3, 4, 4] | Output: [1, 3])
47. **Remove duplicates from sorted array in-place.** (Input: [1, 1, 2, 2, 3] | Output: Length = 3, arr = [1, 2, 3])
48. **Linear Search for element X.** (Input: arr = [4, 2, 7, 1], X = 7 | Output: Index 2; X = 5 | Output: -1)
49. **Binary Search on sorted array.** (Input: arr = [1, 3, 5, 7, 9], target = 7 | Output: Index 3)
50. **Bubble Sort implementation.** (Input: [5, 1, 4, 2, 8] | Output: [1, 2, 4, 5, 8])
51. **Selection Sort implementation.** (Input: [64, 25, 12, 22, 11] | Output: [11, 12, 22, 25, 64])
52. **Insertion Sort implementation.** (Input: [12, 11, 13, 5, 6] | Output: [5, 6, 11, 12, 13])
53. **Left rotate array by K positions.** (Input: arr = [1, 2, 3, 4, 5], K = 2 | Output: [3, 4, 5, 1, 2])
54. **Right rotate array by K positions.** (Input: arr = [1, 2, 3, 4, 5], K = 2 | Output: [4, 5, 1, 2, 3])
55. **Check if array is sorted in ascending order.** (Input: [1, 2, 2, 4, 5] | Output: true; Input: [3, 2, 1] | Output: false)
56. **Find missing number in array from 1 to N.** (Input: arr = [1, 2, 4, 5, 6], N = 6 | Output: 3)
57. **Single number (every element appears twice except one).** (Input: [4, 1, 2, 1, 2] | Output: 4)

58. **Majority element (> N/2 occurrences).** (Input: [3, 2, 3] | Output: 3)
59. **Separate even and odd numbers in array.** (Input: [1, 2, 3, 4, 5, 6] | Output: [2, 4, 6, 1, 3, 5])
60. **Merge two sorted arrays.** (Input: arr1 = [1, 3, 5], arr2 = [2, 4, 6] | Output: [1, 2, 3, 4, 5, 6])

Section 4: Two Pointers, Subarrays & Optimization (Q61 – Q80)

61. ---

    Two Sum (indices or pair). (Input: arr = [2, 7, 11, 15], target = 9 | Output: [0, 1])
62. **Three Sum (triplets adding to 0).** (Input: [-1, 0, 1, 2, -1, -4] | Output: [[-1, -1, 2], [-1, 0, 1]])
63. **Move all zeroes to end.** (Input: [0, 1, 0, 3, 12] | Output: [1, 3, 12, 0, 0])
64. **Segregate 0s and 1s.** (Input: [0, 1, 1, 0, 1, 0] | Output: [0, 0, 0, 1, 1, 1])
65. **Sort array of 0s, 1s, and 2s (Dutch National Flag).** (Input: [2, 0, 2, 1, 1, 0] | Output: [0, 0, 1, 1, 2, 2])
66. **Maximum Subarray Sum (Kadane's Algorithm).** (Input: [-2, 1, -3, 4, -1, 2, 1, -5, 4] | Output: 6 ([4, -1, 2, 1]))
67. **Maximum Product Subarray.** (Input: [2, 3, -2, 4] | Output: 6)
68. **Best Time to Buy and Sell Stock (Single transaction).** (Input: [7, 1, 5, 3, 6, 4] | Output: 5)
69. **Best Time to Buy and Sell Stock II (Multiple transactions).** (Input: [7, 1, 5, 3, 6, 4] | Output: 7)
70. **Peak element.** (Input: [1, 2, 3, 1] | Output: Index 2 (value 3))
71. **Trapping Rain Water.** (Input: [0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1] | Output: 6)
72. **Maximum sum subarray of size K (Sliding Window).** (Input: arr = [2, 1, 5, 1, 3, 2], K = 3 | Output: 9)
73. **First negative integer in every window of size K.** (Input: arr = [12, -1, -7, 8, -15, 30, 16, 28], K = 3 | Output: [-1, -1, -7, -15, -15, 0])
74. **Find equilibrium index.** (Input: [-7, 1, 5, 2, -4, 3, 0] | Output: Index 3)
75. **Find leaders in an array.** (Input: [16, 17, 4, 3, 5, 2] | Output: [17, 5, 2])
76. **Longest Consecutive Sequence.** (Input: [100, 4, 200, 1, 3, 2] | Output: 4 ([1, 2, 3, 4]))
77. **Intersection of two arrays.** (Input: arr1 = [1, 2, 2, 1], arr2 = [2, 2] | Output: [2])
78. **Product of array except self.** (Input: [1, 2, 3, 4] | Output: [24, 12, 8, 6])

79. **Subarray sum equals K.** (Input: arr = [1, 1, 1], K = 2 | Output: 2)
80. **Container with Most Water.** (Input: [1, 8, 6, 2, 5, 4, 8, 3, 7] | Output: 49)

Section 5: 2D Arrays & Matrices (Q81 – Q100)

81. ---

    Matrix sum & element-wise display. (Input: [[1, 2], [3, 4]] | Output: Sum = 10)
82. **Addition of two matrices.** (Input: A = [[1, 2], [3, 4]], B = [[5, 6], [7, 8]] | Output: [[6, 8], [10, 12]])
83. **Multiplication of two matrices.** (Input: A = [[1, 2], [3, 4]], B = [[1, 0], [0, 1]] | Output: [[1, 2], [3, 4]])
84. **Transpose of a matrix.** (Input: [[1, 2, 3], [4, 5, 6]] | Output: [[1, 4], [2, 5], [3, 6]])
85. **Main and anti-diagonal sums of a square matrix.** (Input: [[1, 2, 3], [4, 5, 6], [7, 8, 9]] | Output: Main = 15, Anti = 15)
86. **Check if matrix is symmetric.** (Input: [[1, 2], [2, 1]] | Output: true)
87. **Check Identity matrix.** (Input: [[1, 0], [0, 1]] | Output: true)
88. **Row-wise and column-wise sum.** (Input: [[1, 2], [3, 4]] | Output: Rows = [3, 7], Cols = [4, 6])
89. **Rotate matrix 90 degrees clockwise.** (Input: [[1, 2], [3, 4]] | Output: [[3, 1], [4, 2]])
90. **Rotate matrix 90 degrees counter-clockwise.** (Input: [[1, 2], [3, 4]] | Output: [[2, 4], [1, 3]])
91. **Spiral order traversal.** (Input: [[1, 2, 3], [4, 5, 6], [7, 8, 9]] | Output: [1, 2, 3, 6, 9, 8, 7, 4, 5])
92. **Search in row-wise and column-wise sorted matrix.** (Input: matrix = [[1, 4, 7], [2, 5, 8], [3, 6, 9]], target = 5 | Output: Found at (1, 1))
93. **Set Matrix Zeroes.** (Input: [[1, 1, 1], [1, 0, 1], [1, 1, 1]] | Output: [[1, 0, 1], [0, 0, 0], [1, 0, 1]])
94. **Boundary elements traversal.** (Input: [[1, 2, 3], [4, 5, 6], [7, 8, 9]] | Output: 1, 2, 3, 6, 9, 8, 7, 4)
95. **Row with maximum number of 1s in boolean matrix.** (Input: [[0, 1, 1], [1, 1, 1], [0, 0, 1]] | Output: Row 1)
96. **Check Sparse matrix (more than half elements are zero).** (Input: [[0, 0, 3], [0, 0, 0], [1, 0, 0]] | Output: true)
97. **Anti-diagonal traversal (Zigzag).** (Input: [[1, 2], [3, 4]] | Output: [[1], [2, 3], [4]])
98. **Saddle point of a matrix.** (Input: [[1, 2, 3], [4, 5, 6], [7, 8, 9]] | Output: 7)
99. **Valid Sudoku board verification (9x9).** (Input: 9x9 board configuration | Output: true / false)

100. **Generate N x N spiral matrix.** (Input: N = 3 | Output: [[1, 2, 3], [8, 9, 4], [7, 6, 5]])